



Assignment - 1

Time

```
#include <iostream>
#include <cstdio>

using namespace std;

class Time {
    int days, hour, minute, second;

public:
    Time() {
        days = 0, hour = 0, minute = 0, second = 0;
    }

    Time(int x, int y, int z) {
        hour = x;
        minute = y;
        second = z;
    }

    void display();
    Time add(Time T2);
    Time sub(Time T2);
};

Time Time::add(Time T2){
    Time temp;

    temp.second = (second + T2.second) % 60;
    temp.minute = (second + T2.second) / 60;

    temp.minute += minute + T2.minute;
    temp.minute = temp.minute % 60;

    temp.hour = (minute + T2.minute) / 60;
    temp.hour += hour + T2.hour;

    return temp;
}

Time Time::sub(Time T2) {
    Time temp;
```



```
int seconds_1 = ( hour*60*60 + minute * 60 + second );
int seconds_2 = ( T2.hour*60*60 + T2.minute * 60 + T2.second );
int subs;

if (seconds_1 < seconds_2) {
    subs = seconds_2 - seconds_1;
} else {
    subs = seconds_1 - seconds_2;
}

temp.hour = subs / 3600;
subs %= 3600;
temp.minute = subs / 60;
subs %= 60;
temp.second = subs;

return temp;
}

void pad(int number){
    if (number / 10 == 0) {
        cout << "0" << number;
    } else {
        cout << number;
    }
}

void Time::display() {
    pad(hour) ; cout << ":" ; pad(minute) ; cout << ":" ; pad(second) ; cout
<< endl;
}

int main() {
    Time T1(15,25,45);
    Time T2(6,55,50);

    Time T3, T4;

    T3 = T1.add(T2);
    T4 = T1.sub(T2);

    cout << "1st time: " ; T1.display();
    cout << "2nd time: " ; T2.display();
    cout << "Addition: " ; T3.display();
    cout << "Substraction: " ; T4.display();
}
```



Output:->

1st time: 15:25:45

2nd time: 06:55:50

Addition: 22:21:35

Substraction: 08:29:55

Assignment - 2

Complex number

```
/*
A-3 :-> Create a class complex that contains two data members , real and
imaginary of float type
        to store the real and imaginary part of complex number, the class
        also coantins the following functions :->

        - addition: It takes one complex object as argument and returns the n
        object conatining the addition of them.
        - substraction: --- sub.
        - multiplication: --- mul.
        - division: --- div.

        Write the main fucntion to implement the complex class
*/

#include <cstdio>
#include <iostream>

using namespace std;
```



```
class Complex {
    float real;
    float img;

    public:
        void display();
        void setData(float x, float y);

        Complex add(Complex c2);
        Complex sub(Complex c2);
        Complex mul(Complex c2);
        Complex div(Complex c2);
};

void Complex::setData(float x, float y){
    real = x;
    img = y;
}

void Complex::display(){

    // real check
    /*
    switch (real) {
        case 0:
            case
    }
    */
    if (real != 0){
        cout << real;
    }

    if (img != 0){
        if (img > 0){
            if (real != 0){
                cout << '+';
            }
            cout << img << 'i' << endl;

        } else {
            cout << img << 'i' << endl;
        }
    } else cout << endl;
}

Complex Complex::add(Complex c2){
    Complex temp;
```



```
temp.real = real + c2.real;
temp.img = img + c2.img;

return temp;
}

Complex Complex::sub(Complex c2){
    Complex temp;
    temp.real = real - c2.real;
    temp.img = img - c2.img;

    return temp;
}

Complex Complex::mul(Complex c2){
    Complex temp;

    temp.real = real * c2.real - img * c2.img;
    temp.img = real * c2.img + img * c2.real;

    return temp;
}

Complex Complex::div(Complex c2){
    Complex temp;

    float a = real;
    float b = img;
    float c = c2.real;
    float d = c2.img;

    float base = c*c + d*d;

    temp.real = (a*c + b*d) / base;
    temp.img = (b*c - a*d) / base;

    return temp;
}

int main(){
    Complex c1, c2, c3, c4, c5, c6;

    float x, y;

    cout << "Enter 1st complex number:->" << endl << "Real: ";
    cin >> x;
```



```
cout << "Imaginary: ";
cin >> y;
cout << endl;

// x = 0, y = 3;
c1.setData(x, y);

cout << "Enter 2nd complex number:->" << endl << "Real: ";
cin >> x;
cout << "Imaginary: ";
cin >> y;
cout << endl;

// x = 0, y = 5;
c2.setData(x, y);

c3 = c1.add(c2);
c4 = c1.sub(c2);
c5 = c1.mul(c2);
c6 = c1.div(c2);

cout << "1st complex number: "; c1.display();
cout << "2nd complex number: "; c2.display(); cout << endl;

cout << "Addition: "; c3.display();
cout << "Subtraction: "; c4.display();
cout << "Multiplication: "; c5.display();
cout << "Division: "; c6.display();
}
```

Output:->

--- Example 1: Standard complex numbers (3+4i and 1+2i) ---

Enter 1st complex number:->

Real: Imaginary:

Enter 2nd complex number:->

Real: Imaginary:

1st complex number: 3+4i

2nd complex number: 1+2i



Addition: $4+6i$

Subtraction: $2+2i$

Multiplication: $-5+10i$

Division: $2.2-0.4i$

--- Example 2: Negative and zero values ($5-2i$ and $0+3i$) ---

Enter 1st complex number:->

Real: Imaginary:

Enter 2nd complex number:->

Real: Imaginary:

1st complex number: $5-2i$

2nd complex number: $3i$

Addition: $5+1i$

Subtraction: $5-5i$

Multiplication: $6+15i$

Division: $-0.666667-1.66667i$



Assignment - 3

Complex number using operator overloading

```
/*
A-3 :-> Create a class complex that contains two data members , real and
imaginary of float type
        to store the real and imaginary part of complex number, the class
        also contains the following functions :->

        - addition: It takes one complex object as argument and returns the n
        object containing the addition of them.
        - subtraction: --- sub.
        - multiplication: --- mul.
        - division: --- div.

        via overloading operators

        Write the main function to implement the complex class
*/

#include <stdio.h>
#include <iostream>

using namespace std;

class Complex {
    float real;
    float img;

    public:
        void display();
        void setData(float x, float y);

        Complex operator+(Complex c2);
        Complex operator-(Complex c2);
        Complex operator*(Complex c2);
        Complex operator/(Complex c2);
};

void Complex::setData(float x, float y){
    real = x;
    img = y;
}
```




```
}

void Complex::display(){

    // real check
    /*
    switch (real) {
        case 0:
            case
    }
    */
    if (real != 0){
        cout << real;
    }

    if (img != 0){
        if (img > 0){
            if (real != 0){
                cout << '+';
            }
            cout << img << 'i' << endl;

        } else {
            cout << img << 'i' << endl;
        }
    } else cout << endl;
}

Complex Complex::operator+(Complex c2){
    Complex temp;
    temp.real = real + c2.real;
    temp.img = img + c2.img;

    return temp;
}

Complex Complex::operator-(Complex c2){
    Complex temp;
    temp.real = real - c2.real;
    temp.img = img - c2.img;

    return temp;
}

Complex Complex::operator*(Complex c2){
    Complex temp;

    temp.real = real * c2.real - img * c2.img;
```



```
temp.img = real * c2.img + img * c2.real;

return temp;
}

Complex Complex::operator/(Complex c2){
    Complex temp;

    float a = real;
    float b = img;
    float c = c2.real;
    float d = c2.img;

    float base = c*c + d*d;

    temp.real = (a*c + b*d) / base;
    temp.img = (b*c - a*d) / base;

    return temp;
}

int main(){
    Complex c1, c2, c3, c4, c5, c6;

    float x, y;

    cout << "Enter 1st complex number:->" << endl << "Real: ";
    cin >> x;
    cout << "Imaginary: ";
    cin >> y;
    cout << endl;

    // x = 0, y = 3;
    c1.setData(x, y);

    cout << "Enter 2nd complex number:->" << endl << "Real: ";
    cin >> x;
    cout << "Imaginary: ";
    cin >> y;
    cout << endl;

    // x = 0, y = 5;
    c2.setData(x, y);

    c3 = c1 + c2;
```



```
c4 = c1 - c2;
c5 = c1 * c2;
c6 = c1 / c2;

cout << "1st complex number: "; c1.display();
cout << "2nd complex number: "; c2.display(); cout << endl;

cout << "Addition: "; c3.display();
cout << "Substraction: "; c4.display();
cout << "Multiplication: "; c5.display();
cout << "Division: "; c6.display();
}
```

Output:->

--- Example 1: Standard complex numbers (8+3i and 4-2i) ---

Enter 1st complex number:->

Real: Imaginary:

Enter 2nd complex number:->

Real: Imaginary:

1st complex number: 8+3i

2nd complex number: 4-2i

Addition: 12+1i

Substraction: 4+5i

Multiplication: 38-4i

Division: 1.3+1.4i

--- Example 2: Division with fractional results (3+2i and 4+1i) ---

Enter 1st complex number:->

Real: Imaginary:

Enter 2nd complex number:->

Real: Imaginary:

1st complex number: 3+2i

2nd complex number: 4+1i



Addition: $7+3i$

Subtraction: $-1+i$

Multiplication: $10+11i$

Division: $0.823529+0.294118i$

Assignment - 4

Complex number using operator overloading + friend function

```
/*
A-3 :-> Create a class complex that contains two data members , real and
imaginary of float type
        to store the real and imaginary part of complex number, the class
        also contains the following functions :->

        - addition: It takes one complex object as argument and returns the n
        object containing the addition of them.
        - subtraction: --- sub.
        - multiplication: --- mul.
        - division: --- div.

        via overloading operators + friend function

        Write the main function to implement the complex class
*/

#include <stdio>
#include <iostream>

using namespace std;

class Complex {
    float real;
    float img;
```



```
public:
    void display();
    void setData(float x, float y);

    friend Complex operator+(Complex c1, Complex c2);
    friend Complex operator-(Complex c1, Complex c2);
    friend Complex operator*(Complex c1, Complex c2);
    friend Complex operator/(Complex c1, Complex c2);
};

void Complex::setData(float x, float y){
    real = x;
    img = y;
}

void Complex::display(){

    // real check
    /*
    switch (real) {
        case 0:
            case
    }
    */
    if (real != 0){
        cout << real;
    }

    if (img != 0){
        if (img > 0){
            if (real != 0){
                cout << '+';
            }
            cout << img << 'i' << endl;
        } else {
            cout << img << 'i' << endl;
        }
    } else cout << endl;
}

Complex operator+(Complex c1, Complex c2){
    Complex temp;
    temp.real = c1.real + c2.real;
    temp.img = c1.img + c2.img;

    return temp;
}
```



```
Complex operator-(Complex c1, Complex c2){
    Complex temp;
    temp.real = c1.real - c2.real;
    temp.img = c1.img - c2.img;

    return temp;
}

Complex operator*(Complex c1, Complex c2){
    Complex temp;

    temp.real = c1.real * c2.real - c1.img * c2.img;
    temp.img = c1.real * c2.img + c1.img * c2.real;

    return temp;
}

Complex operator/(Complex c1, Complex c2){
    Complex temp;

    float a = c1.real;
    float b = c1.img;
    float c = c2.real;
    float d = c2.img;

    float base = c*c + d*d;

    temp.real = (a*c + b*d) / base;
    temp.img = (b*c - a*d) / base;

    return temp;
}

int main(){
    Complex c1, c2, c3, c4, c5, c6;

    float x, y;

    cout << "Enter 1st complex number:->" << endl << "Real: ";
    cin >> x;
    cout << "Imaginary: ";
    cin >> y;
    cout << endl;

    // x = 0, y =3;
```



```
c1.setData(x, y);

cout << "Enter 2nd complex number:->" << endl << "Real: ";
cin >> x;
cout << "Imaginary: ";
cin >> y;
cout << endl;

// x = 0, y = 5;
c2.setData(x, y);

c3 = c1 + c2;
c4 = c1 - c2;
c5 = c1 * c2;
c6 = c1 / c2;

cout << "1st complex number: "; c1.display();
cout << "2nd complex number: "; c2.display(); cout << endl;

cout << "Addition: "; c3.display();
cout << "Substraction: "; c4.display();
cout << "Multiplication: "; c5.display();
cout << "Division: "; c6.display();
}
```

Output:->

--- Example 1: Standard complex numbers (10+2i and 5+8i) ---

Enter 1st complex number:->

Real: Imaginary:

Enter 2nd complex number:->

Real: Imaginary:

1st complex number: 10+2i

2nd complex number: 5+8i

Addition: 15+10i

Substraction: 5-6i



Multiplication: $34+90i$

Division: $0.741573-0.786517i$

--- Example 2: Numbers with negative parts ($-4-2i$ and $3-7i$) ---

Enter 1st complex number:->

Real: Imaginary:

Enter 2nd complex number:->

Real: Imaginary:

1st complex number: $-4-2i$

2nd complex number: $3-7i$

Addition: $-1-9i$

Subtraction: $-7+5i$

Multiplication: $-26+22i$

Division: $0.0344828-0.586207i$



Assignment - 5

Matrix

```
/*
Construct a class matrix with requisite data members write a c++ program
using appropriate member functions to add, subtract & multiply two given
matrices
*/

#include <iostream>
#include <cstdio>

using namespace std;

class Matrix {
    int matrix[10][10];
    int rows, columns;

public:
    Matrix(){};
    Matrix(int r, int c){
        rows = r;
        columns = c;
    }

    Matrix add(Matrix M2);
    Matrix sub(Matrix M2);
    Matrix mul(Matrix M2);
    void read();
    void display();
};

Matrix Matrix::add(Matrix M2){
    Matrix temp;

    // check
    if ( (rows != M2.rows) || (columns != M2.columns) ) {
        cout << "Matrix size mismatch\n";
        // return 1;
    }

    temp.rows = rows;
    temp.columns = columns;
```



```
for (int r = 0; r < rows ; r++) {
    for (int c = 0; c < columns ; c++){
        temp.matrix[r][c] = matrix[r][c] + M2.matrix[r][c];
    }
}

return temp;
}

Matrix Matrix::sub(Matrix M2){
    Matrix temp;

    // check
    if ( (rows != M2.rows) || (columns != M2.columns) ) {
        cout << "Matrix size mismatch\n";
        // return 1;
    }

    temp.rows = rows;
    temp.columns = columns;

    for (int r = 0; r < rows ; r++) {
        for (int c = 0; c < columns ; c++){
            temp.matrix[r][c] = matrix[r][c] - M2.matrix[r][c];
        }
    }

    return temp;
}

Matrix Matrix::mul(Matrix M2){
    // check
    if (columns != M2.rows) {
        cout << "Matrix multiplication not possible,\n M1.columns !=
M2.rows\n";
        // return 1;
    }

    Matrix temp;
    temp.rows = rows;
    temp.columns = M2.columns;

    for (int i = 0 ; i < temp.rows; i++){
        for (int j = 0 ; j < temp.columns ; j++) {

            int rc = 0;
            for (int k = 0 ; k < columns ; k++) {
```



```
        rc += matrix[i][k]*M2.matrix[k][j];
    }

    temp.matrix[i][j] = rc;

}

}

return temp;

}

void Matrix::display(){
    for (int r = 0; r < rows ; r++) {
        for (int c = 0; c < columns ; c++){
            cout << matrix[r][c] << ' ';
        }

        cout << endl;
    }
}

void Matrix::read(){
    cout << "Enter rows and columns: ";
    cin >> rows >> columns;
    cout << endl;

    for (int r = 0; r < rows ; r++) {
        printf("Row %d: ", r);
        for (int c = 0; c < columns ; c++){
            cin >> matrix[r][c];
        }

    }

}

int main(){

    // Matrix M1(2 , 2);
    // Matrix M2(2 , 2);

    Matrix M1, M2;

    cout << "Enter 1st matrix: \n"; M1.read(); cout << endl ;
    cout << "Enter 2nd matrix: \n"; M2.read(); cout << endl ;
```



```
Matrix sum, sub, mul;
sum = M1.add(M2);
sub = M1.sub(M2);
mul = M1.mul(M2);

cout << "Matrix 1:-> \n" ; M1.display() ; cout << endl;
cout << "Matrix 2:-> \n" ; M2.display() ; cout << endl;
cout << "Addition :-> \n" ; sum.display() ; cout << endl;
cout << "Substraction :-> \n" ; sub.display() ; cout << endl;
cout << "Multiplicatiuon :-> \n" ; mul.display() ; cout << endl;
}
```

Output:->

--- Example 1: All operations valid (2x2 matrices) ---

Enter 1st matrix:

Enter rows and columns:

Row 0: Row 1:

Enter 2nd matrix:

Enter rows and columns:

Row 0: Row 1:

Matrix 1:->

1 2

3 4

Matrix 2:->

5 6

7 8

Addition :->

6 8

10 12

Substraction :->



-4 -4

-4 -4

Multiplicatiuon :->

19 22

43 50

--- Example 2: Add/Sub valid, Mul invalid (two 2x3 matrices) ---

Enter 1st matrix:

Enter rows and columns:

Row 0: Row 1:

Enter 2nd matrix:

Enter rows and columns:

Row 0: Row 1:

Matrix multiplication not possible,

$M1.columns \neq M2.rows$

Matrix 1:->

1 2 3

4 5 6

Matrix 2:->

7 8 9

1 2 3

Addition :->

8 10 12

5 7 9

Substraction :->

-6 -6 -6

3 3 3



Assignment - 6

Matrix using friend function

```
/*
Construct a class matrix with requisite data members write a c++ program
using appropriate member functions to add, subtract & multiply two given
matrices
*/

#include <iostream>
#include <cstdio>

using namespace std;

class Matrix {
    int matrix[10][10];
    int rows, columns;

public:
    Matrix(){};
    Matrix(int r, int c){
        rows = r;
        columns = c;
    }

    friend Matrix operator+(Matrix M1, Matrix M2);
    friend Matrix operator-(Matrix M1, Matrix M2);
    friend Matrix operator*(Matrix M1, Matrix M2);

    void read();
    void display();
};

Matrix operator+(Matrix M1, Matrix M2){
    Matrix temp;

    // check
    if ( (M1.rows != M2.rows) || (M1.columns != M2.columns) ) {
        cout << "Matrix size mismatch\n";
        // return 1;
    }

    temp.rows = M1.rows;
    temp.columns = M1.columns;
```



```
for (int r = 0; r < M1.rows ; r++) {
    for (int c = 0; c < M1.columns ; c++){
        temp.matrix[r][c] = M1.matrix[r][c] + M2.matrix[r][c];
    }
}

return temp;
}

Matrix operator-(Matrix M1, Matrix M2){
    Matrix temp;

    // check
    if ( (M1.rows != M2.rows) || (M1.columns != M2.columns) ) {
        cout << "Matrix size mismatch\n";
        // return 1;
    }

    temp.rows = M1.rows;
    temp.columns = M1.columns;

    for (int r = 0; r < M1.rows ; r++) {
        for (int c = 0; c < M1.columns ; c++){
            temp.matrix[r][c] = M1.matrix[r][c] - M2.matrix[r][c];
        }
    }

    return temp;
}

Matrix operator*(Matrix M1, Matrix M2){
    // check
    if (M1.columns != M2.rows) {
        cout << "Matrix multiplication not possible,\n M1.columns !=
M2.rows\n";
        // return 1;
    }

    Matrix temp;
    temp.rows = M1.rows;
    temp.columns = M2.columns;

    for (int i = 0 ; i < temp.rows; i++){
        for (int j = 0 ; j < temp.columns ; j++) {

            int rc = 0;
            for (int k = 0 ; k < M1.columns ; k++) {
```



```
        rc += M1.matrix[i][k]*M2.matrix[k][j];
    }

    temp.matrix[i][j] = rc;

}

}

return temp;

}

void Matrix::display(){
    for (int r = 0; r < rows ; r++) {
        for (int c = 0; c < columns ; c++){
            cout << matrix[r][c] << ' ';
        }

        cout << endl;
    }
}

void Matrix::read(){
    cout << "Enter no. of rows and columns: ";
    cin >> rows >> columns;
    cout << endl;

    for (int r = 0; r < rows ; r++) {
        printf("Row %d: ", r);
        for (int c = 0; c < columns ; c++){
            cin >> matrix[r][c];
        }

    }

}

int main(){

    // Matrix M1(2 , 2);
    // Matrix M2(2 , 2);

    Matrix M1, M2;

    cout << "Enter 1st matrix: \n"; M1.read(); cout << endl ;
    cout << "Enter 2nd matrix: \n"; M2.read(); cout << endl ;
```




```
Matrix sum, sub, mul;
sum = M1 + M2;
sub = M1 - M2;
mul = M1 * M2;

cout << "Matrix 1:-> \n" ; M1.display() ; cout << endl;
cout << "Matrix 2:-> \n" ; M2.display() ; cout << endl;
cout << "Addition :-> \n" ; sum.display() ; cout << endl;
cout << "Substraction :-> \n" ; sub.display() ; cout << endl;
cout << "Multiplicatiuon :-> \n" ; mul.display() ; cout << endl;
}
```

Output:->

--- Example 1: All operations valid (3x3 matrices) ---

Enter 1st matrix:

Enter no. of rows and columns:

Row 0: Row 1: Row 2:

Enter 2nd matrix:

Enter no. of rows and columns:

Row 0: Row 1: Row 2:

Matrix 1:->

1 1 1

2 2 2

3 3 3

Matrix 2:->

9 9 9

8 8 8

7 7 7

Addition :->



10 10 10

10 10 10

10 10 10

Substraction :->

-8 -8 -8

-6 -6 -6

-4 -4 -4

Multiplicatiuon :->

24 24 24

48 48 48

72 72 72

--- Example 2: Mul valid, Add/Sub invalid (2x3 and 3x2) ---

Enter 1st matrix:

Enter no. of rows and columns:

Row 0: Row 1:

Enter 2nd matrix:

Enter no. of rows and columns:

Row 0: Row 1: Row 2:

Matrix size mismatch

Matrix size mismatch

Matrix 1:->



1 2 3

4 5 6

Matrix 2:->

7 8

9 1

2 3

Addition :->

8 10 59

13 6 1070548734

Substraction :->

-6 -6 -53

-5 4 -1070548722

Multiplicatiuon :->

31 19

85 55



Assignment - 7

Stack using array

```
#include <iostream>

using namespace std;
const int SIZE = 5;

class Stack
{
    int stack[SIZE];
    int top = -1;

    public:
        void push(int n) {
            if (top >= SIZE-1) {
                cout << "Stack is full\n";
            } else {
                top++;
                stack[top] = n;
            }
        }

        int pop();
        void display();
};

int Stack::pop() {
    if (top < 0) {
        cout << "Stack is empty\n";
        return 0;
    } else
    {
        int n = stack[top--];
        return n;
    }
}

void Stack::display() {
    if (top < 0) {
        cout << "Empty stack !!!\n";
        return;
    }
}
```



```
for (int i = 0; i <= top ; i++) {
    cout << stack[i] << " ";
}
cout << endl;
}

int main() {
    Stack stack;
    int choice;

    while (1) {
        cout << "1. Push \n2. Pop \n3. Display \n4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {

            case 1: {
                int n;
                cout << "Enter element: ";
                cin >> n ;
                stack.push(n);
                break;
            }

            case 2: {
                int pop = stack.pop();
                cout << "Popped element: " << pop << endl;
                cout << "Stack after pop: ";
                stack.display();
                break;
            }

            case 3: {
                stack.display();
                break;
            }

            case 4:
                return 0;

        }
    }
}
```



Output:->

--- Abridged Stack Demonstration ---

1. Push
2. Pop
3. Display
4. Exit

Enter choice: Enter element: 1. Push

2. Pop
3. Display
4. Exit

Enter choice: Enter element: 1. Push

2. Pop
3. Display
4. Exit

Enter choice: 10 20

1. Push
2. Pop
3. Display
4. Exit

Enter choice: Popped element: 20

Stack after pop: 10

1. Push
2. Pop
3. Display
4. Exit

Enter choice: 10



1. Push
2. Pop
3. Display
4. Exit

Enter choice: Popped element: 10

Stack after pop: Empty stack !!!

1. Push
2. Pop
3. Display
4. Exit

Enter choice: Stack is empty

Popped element: 0

Stack after pop: Empty stack !!!

1. Push
2. Pop
3. Display
4. Exit

Enter choice:



Assignment - 8

Stack using linked list

```
#include <iostream>

using namespace std;

// Node structure for the Linked List
struct Node {
    int data;
    Node* next;
};

// Define Nodeptr for convenience
typedef Node* Nodeptr;

class Stack {
    Nodeptr top;

public:
    // Constructor to initialize an empty stack
    Stack() {
        top = NULL;
    }

    void push(int n);
    int pop();
    void display();
};

void Stack::push(int n) {
    // Create a new node
    Nodeptr newNode = new Node;
    newNode->data = n;

    // Link the new node to the top of the stack
    newNode->next = top;

    // Update top to the new node
    top = newNode;
}

int Stack::pop() {
    if (top == NULL) {
        cout << "Stack is empty\n";
    }
}
```




```
        return 0; // Return a default value for an empty stack
    }

    // Get the data from the top node
    int n = top->data;

    // Move top to the next node
    Nodeptr temp = top;
    top = top->next;

    // Free the old top node
    delete temp;

    return n;
}

void Stack::display() {
    if (top == NULL) {
        cout << "Empty stack !!!\n";
        return;
    }

    Nodeptr current = top;
    cout << "Stack (top to bottom): ";
    while (current != NULL) {
        cout << current->data << " ";
        current = current->next;
    }
    cout << endl;
}

int main() {
    Stack stack;
    int choice;

    while (1) {
        cout << "1. Push \n2. Pop \n3. Display \n4. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {

            case 1: {
                int n;
                cout << "Enter element: ";
                cin >> n;
                stack.push(n);
                break;
            }
        }
    }
}
```



```
    }

    case 2: {
        int popped_val = stack.pop();
        if (popped_val != 0) { // A simple check to see if pop was
successful
            cout << "Popped element: " << popped_val << endl;
            cout << "Stack after pop: ";
            stack.display();
        }
        break;
    }

    case 3: {
        stack.display();
        break;
    }

    case 4:
        return 0;

    default:
        cout << "Invalid choice! Please try again." << endl;
    }
}
}
```

Output:->

--- Abridged Linked List Stack Demonstration ---

1. Push
 2. Pop
 3. Display
 4. Exit
- Enter choice: Enter element: 1. Push
2. Pop
 3. Display
 4. Exit



Enter choice: Enter element: 1. Push

2. Pop

3. Display

4. Exit

Enter choice: Stack (top to bottom): 200 100

1. Push

2. Pop

3. Display

4. Exit

Enter choice: Popped element: 200

Stack after pop: Stack (top to bottom): 100

1. Push

2. Pop

3. Display

4. Exit

Enter choice: Stack (top to bottom): 100

1. Push

2. Pop

3. Display

4. Exit

Enter choice: Popped element: 100

Stack after pop: Empty stack !!!

1. Push

2. Pop

3. Display

4. Exit

Enter choice: Stack is empty



1. Push
2. Pop
3. Display
4. Exit

Enter choice:

Assignment - 9

Linear Linked List

```
/*  
A-9:-> Write a program using class to perform the following operations on a  
Linear linked list  
  
    i) Insertion  
    ii) deletion  
    iii) display  
    iv) sorting  
    v) reverse  
    vi) searching  
    vii) concatenation  
*/  
  
#include <iostream>  
  
using namespace std;  
  
struct Node {  
    int data;  
    struct Node* next;  
};  
typedef struct Node* Nodeptr;  
  
class LinearLL {  
    Nodeptr head;  
  
public:  
    // Default constructor
```



```
LinearLL() {
    head = NULL;
}

// Destructor to free memory
~LinearLL() {
    Nodeptr current = head;
    while (current != NULL) {
        Nodeptr next = current->next;
        delete current;
        current = next;
    }
}

void insert(int data, int position);
void del(int position);
void display();
void sort();
void reverse();
int search(int data);
void concat(LinearLL& list2);
void append(int data); // Helper to add to the end
};

void LinearLL::append(int data) {
    Nodeptr newNode = new Node;
    newNode->data = data;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        Nodeptr p = head;
        while (p->next != NULL) {
            p = p->next;
        }
        p->next = newNode;
    }
}

void LinearLL::display() {
    if (head == NULL) {
        cout << "List is empty." << endl;
        return;
    }
    Nodeptr nxt = head;
    while (nxt != NULL) {
        cout << nxt->data << " -> ";
    }
}
```



```
        nxt = nxt->next;
    }
    cout << "NULL" << endl;
}

void LinearLL::insert(int data, int pos) {
    Nodeptr newNode = new Node;
    newNode->data = data;

    if (pos <= 0) {
        cout << "Invalid position!" << endl;
        delete newNode;
        return;
    }

    if (pos == 1) {
        newNode->next = head;
        head = newNode;
        return;
    }

    Nodeptr p = head;
    for (int i = 1; i < pos - 1 && p != NULL; i++) {
        p = p->next;
    }

    if (p == NULL) {
        cout << "Position out of bounds!" << endl;
        delete newNode;
    } else {
        newNode->next = p->next;
        p->next = newNode;
    }
}

void LinearLL::del(int pos) {
    if (head == NULL) {
        cout << "List is empty." << endl;
        return;
    }
    if (pos <= 0) {
        cout << "Invalid position!" << endl;
        return;
    }

    Nodeptr temp;
    if (pos == 1) {
        temp = head;
```



```
        head = head->next;
        delete temp;
        return;
    }

    Nodeptr p = head;
    for (int i = 1; i < pos - 1 && p->next != NULL; i++) {
        p = p->next;
    }

    if (p->next == NULL) {
        cout << "Position out of bounds!" << endl;
    } else {
        temp = p->next;
        p->next = temp->next;
        delete temp;
    }
}

void LinearLL::sort() {
    if (head == NULL || head->next == NULL) return;

    for (Nodeptr p = head; p->next != NULL; p = p->next) {
        for (Nodeptr q = p->next; q != NULL; q = q->next) {
            if (p->data > q->data) {
                int temp = p->data;
                p->data = q->data;
                q->data = temp;
            }
        }
    }
}

void LinearLL::reverse() {
    if (head == NULL || head->next == NULL) return;

    Nodeptr prev = NULL, current = head, next = NULL;
    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    head = prev;
}

int LinearLL::search(int data) {
    Nodeptr p = head;
```



```
int pos = 1;
while (p != NULL) {
    if (p->data == data) {
        return pos;
    }
    p = p->next;
    pos++;
}
return -1; // Not found
}

void LinearLL::concat(LinearLL& list2) {
    if (list2.head == NULL) return;

    if (head == NULL) {
        head = list2.head;
    } else {
        Nodeptr p = head;
        while (p->next != NULL) {
            p = p->next;
        }
        p->next = list2.head;
    }
    // Prevent list2 from deleting the concatenated nodes
    list2.head = NULL;
}

int main() {
    LinearLL list1;
    int choice, data, pos;

    while (true) {
        cout << "\n--- Linear Linked List Menu ---" << endl;
        cout << "1. Append (add to end)" << endl;
        cout << "2. Insert at position" << endl;
        cout << "3. Delete from position" << endl;
        cout << "4. Display list" << endl;
        cout << "5. Sort list" << endl;
        cout << "6. Reverse list" << endl;
        cout << "7. Search for element" << endl;
        cout << "8. Concatenate with a new list" << endl;
        cout << "9. Exit" << endl;
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter data to append: ";
```




```
        cin >> data;
        list1.append(data);
        cout << "Current list: ";
        list1.display();
        break;
    case 2:
        cout << "Enter data to insert: ";
        cin >> data;
        cout << "Enter position: ";
        cin >> pos;
        list1.insert(data, pos);
        cout << "Current list: ";
        list1.display();
        break;
    case 3:
        cout << "Enter position to delete: ";
        cin >> pos;
        list1.del(pos);
        cout << "Current list: ";
        list1.display();
        break;
    case 4:
        list1.display();
        break;
    case 5:
        list1.sort();
        cout << "Sorted list: ";
        list1.display();
        break;
    case 6:
        list1.reverse();
        cout << "Reversed list: ";
        list1.display();
        break;
    case 7:
        cout << "Enter data to search for: ";
        cin >> data;
        pos = list1.search(data);
        if (pos != -1) {
            cout << "Element " << data << " found at position: " <<
pos << endl;
        } else {
            cout << "Element " << data << " not found in the list." <<
endl;
        }
        break;
    case 8: {
        LinearLL list2;
```



```
        int n, val;
        cout << "How many elements in the second list? ";
        cin >> n;
        for (int i = 0; i < n; ++i) {
            cout << "Enter element " << i + 1 << ": ";
            cin >> val;
            list2.append(val);
        }
        cout << "Second list: ";
        list2.display();
        list1.concat(list2);
        cout << "Concatenated list: ";
        list1.display();
        break;
    }
    case 9:
        cout << "Exiting." << endl;
        return 0;
    default:
        cout << "Invalid choice. Please try again." << endl;
    }
}
return 0;
}
```

Output:->

--- Abridged Linked List Demonstration ---

--- Linear Linked List Menu ---

1. Append (add to end)
2. Insert at position
3. Delete from position
4. Display list
5. Sort list
6. Reverse list
7. Search for element
8. Concatenate with a new list



9. Exit

Enter your choice: Enter data to append: Current list: 10 -> NULL

--- Linear Linked List Menu ---

1. Append (add to end)

2. Insert at position

3. Delete from position

4. Display list

5. Sort list

6. Reverse list

7. Search for element

8. Concatenate with a new list

9. Exit

Enter your choice: Enter data to append: Current list: 10 -> 30 -> NULL

--- Linear Linked List Menu ---

1. Append (add to end)

2. Insert at position

3. Delete from position

4. Display list

5. Sort list

6. Reverse list

7. Search for element

8. Concatenate with a new list

9. Exit

Enter your choice: 10 -> 30 -> NULL



--- Linear Linked List Menu ---

1. Append (add to end)
2. Insert at position
3. Delete from position
4. Display list
5. Sort list
6. Reverse list
7. Search for element
8. Concatenate with a new list
9. Exit

Enter your choice: Enter data to insert: Enter position: Current list: 10 -> 20 -> 30 -> NULL

--- Linear Linked List Menu ---

1. Append (add to end)
2. Insert at position
3. Delete from position
4. Display list
5. Sort list
6. Reverse list
7. Search for element
8. Concatenate with a new list
9. Exit

Enter your choice: 10 -> 20 -> 30 -> NULL

--- Linear Linked List Menu ---

1. Append (add to end)



2. Insert at position
3. Delete from position
4. Display list
5. Sort list
6. Reverse list
7. Search for element
8. Concatenate with a new list
9. Exit

Enter your choice: Enter position to delete: Current list: 20 -> 30 -> NULL

--- Linear Linked List Menu ---

1. Append (add to end)
2. Insert at position
3. Delete from position
4. Display list
5. Sort list
6. Reverse list
7. Search for element
8. Concatenate with a new list
9. Exit

Enter your choice: 20 -> 30 -> NULL

--- Linear Linked List Menu ---

1. Append (add to end)
2. Insert at position
3. Delete from position
4. Display list



5. Sort list
 6. Reverse list
 7. Search for element
 8. Concatenate with a new list
 9. Exit
- Enter your choice: Exiting.

Assignment - 10

String comparison

```
/*  
Write a program using class to compare and concat 2 strings with and without  
friend function  
*/  
  
#include <iostream>  
  
using namespace std;  
  
class String {  
    char data[50];  
  
    public:  
        int operator==(String S2);  
        String operator+(String S2);  
        void display();  
        void read();  
};  
  
int String::operator==(String S2){  
    int i = 0;  
  
    while ( data[i] != '\0') {  
        if ( data[i] != S2.data[i] ) {  
            return 0;  
        }  
    }  
}
```



```
        }
        i++;

    }

    return 1;
}

String String::operator+(String S2) {
    String temp;

    int len = 0;

    while (data[len] != '\0') {
        temp.data[len] = data[len];
        len++;
    }

    int len2 = 0;
    while (S2.data[len2] != '\0') {
        temp.data[len++] = S2.data[len2++];
    }
    temp.data[len] = '\0'; // Null-terminate the concatenated string

    // debug
    // cout << "Length of string: " << len << " " << len2 << endl;
    return temp;
}

void String::display(){
    cout << data << endl;
}

void String::read() {
    cin >> data;
}

int main() {
    String s1, s2, s3;

    cout << " Enter 1st string: "; s1.read();
    cout << " Enter 2nd string: "; s2.read();

    s3 = s1 + s2;
    if ( (s1 == s2) ) {
        cout << "Equal strings" << endl;
    }
}
```



```
    } else {  
        cout << "Unequal strings" << endl;  
    }  
  
    cout << "Concatenated: " ; s3.display();  
  
}
```

Output:->

--- Example 1: Equal strings ('hello' and 'hello') ---

Enter 1st string: Enter 2nd string: Equal strings

Concatenated: hellohello

--- Example 2: Unequal strings ('hello' and 'world') ---

Enter 1st string: Enter 2nd string: Unequal strings

Concatenated: helloworld

Assignment - 11

String comparison using friend function

```
/*  
Write a program using class to compare and concat 2 strings with and without  
friend function  
(FRIEND FUNCTION VERSION)  
*/  
  
#include <iostream>  
#include <cstring> // For strcpy and strcat  
  
using namespace std;
```




```
class String {
    char data[100];

public:
    void display();
    void read();

    // Friend function declarations
    friend int operator==(const String& s1, const String& s2);
    friend String operator+(const String& s1, const String& s2);
};

void String::display() {
    cout << data << endl;
}

void String::read() {
    cin >> data;
}

// Friend function for comparison
int operator==(const String& s1, const String& s2) {
    int i = 0;
    while (s1.data[i] != '\0' || s2.data[i] != '\0') {
        if (s1.data[i] != s2.data[i]) {
            return 0; // Not equal
        }
        i++;
    }
    return 1; // Equal
}

// Friend function for concatenation
String operator+(const String& s1, const String& s2) {
    String temp;
    // Copy first string
    int i = 0;
    while (s1.data[i] != '\0') {
        temp.data[i] = s1.data[i];
        i++;
    }

    // Append second string
    int j = 0;
    while (s2.data[j] != '\0') {
        temp.data[i++] = s2.data[j++];
    }
    temp.data[i] = '\0'; // Null-terminate the result
}
```



```
        return temp;
    }

    int main() {
        String s1, s2, s3;

        cout << "Enter 1st string: ";
        s1.read();
        cout << "Enter 2nd string: ";
        s2.read();

        if (s1 == s2) {
            cout << "Equal strings" << endl;
        } else {
            cout << "Unequal strings" << endl;
        }

        s3 = s1 + s2;
        cout << "Concatenated: ";
        s3.display();

        return 0;
    }
```

Output:->

--- Example 1: Equal strings ('test' and 'test') ---

Enter 1st string: Enter 2nd string: Equal strings

Concatenated: testtest

--- Example 2: Unequal strings ('apple' and 'orange') ---

Enter 1st string: Enter 2nd string: Unequal strings

Concatenated: appleorange